05: Direct Memory Access (DMA)

Engr 315: Hardware / Software Codesign

Andrew Lukefahr
Indiana University

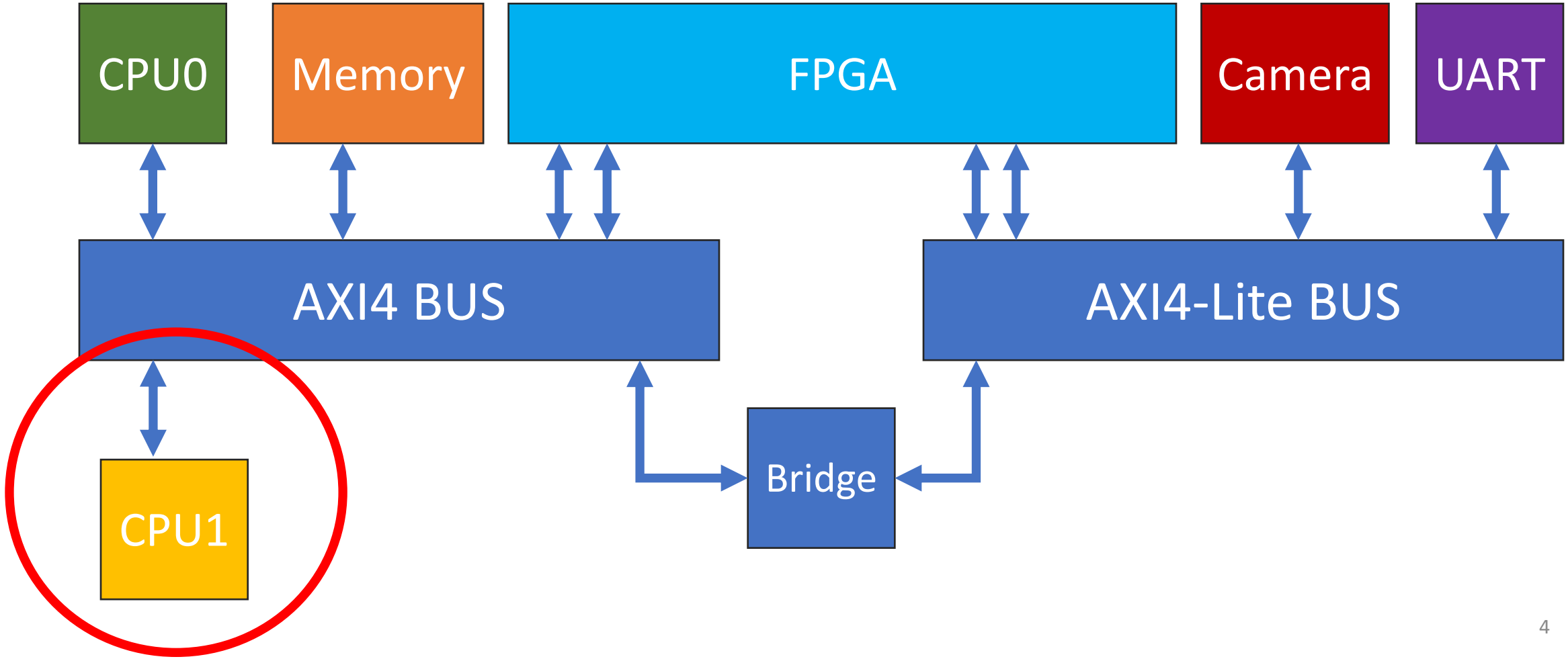
Let’s talk P2

Other Announcements

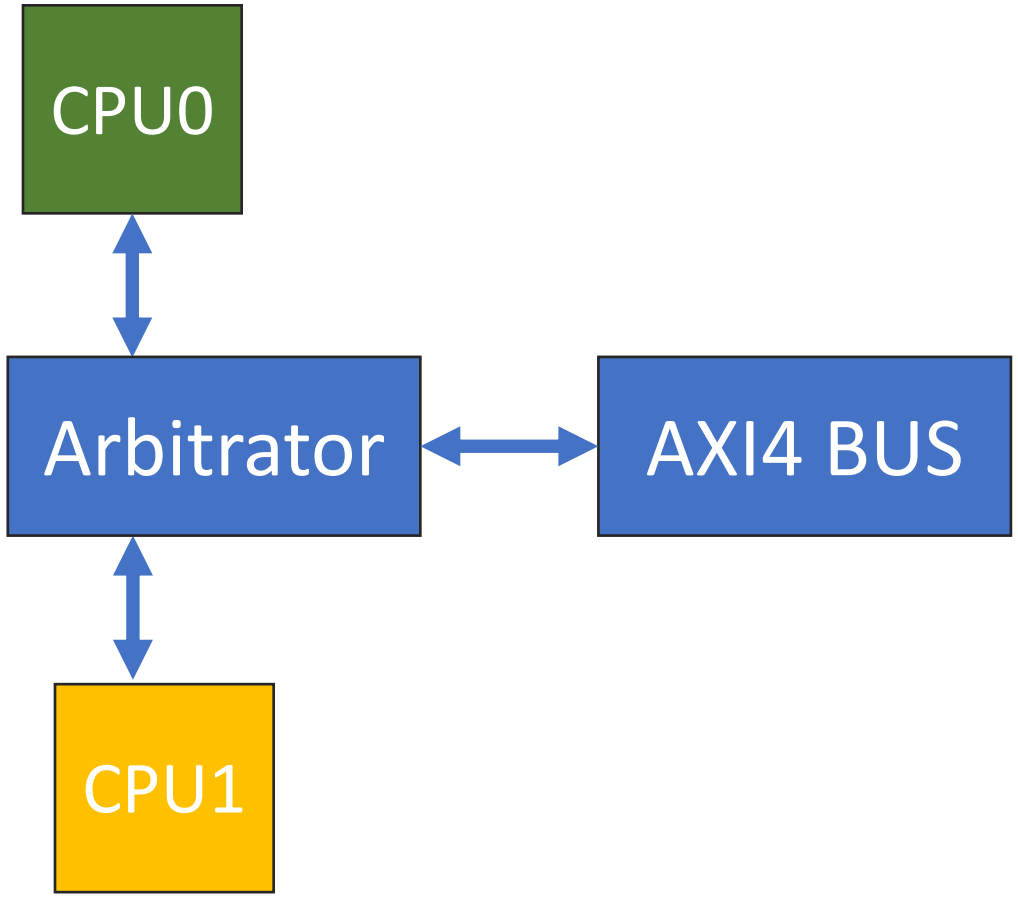
- Slack: Email Zach zwlevy@iu.edu
- Ethan's Office Hours schedule (website)

Wednesday	5.30 – 8:00 pm
Thursday	2.30 – 4:00 pm

Review: Multi-Master Buses



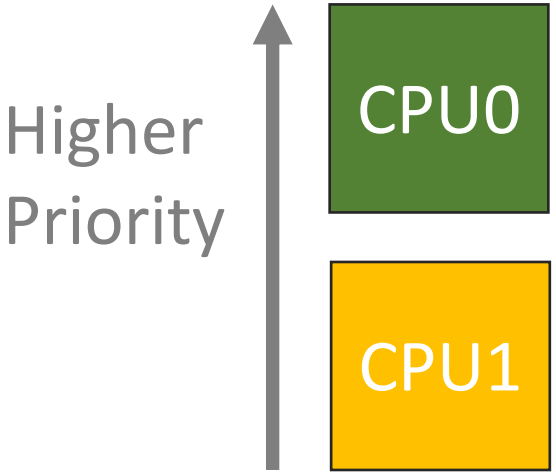
An Arbitrator selects who gets to use the bus



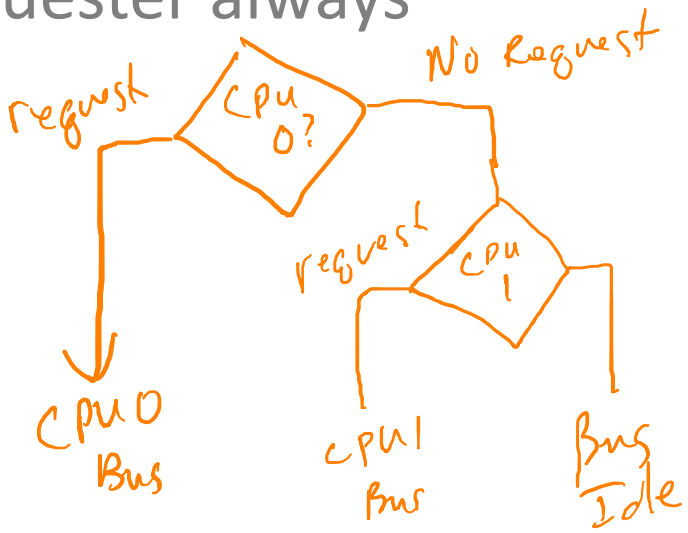
- What happens if both request a transaction at the same time?

- Arbitration:
  - Fixed-Priority
  - Round Robin
  - Many more…

Highest-Priority First

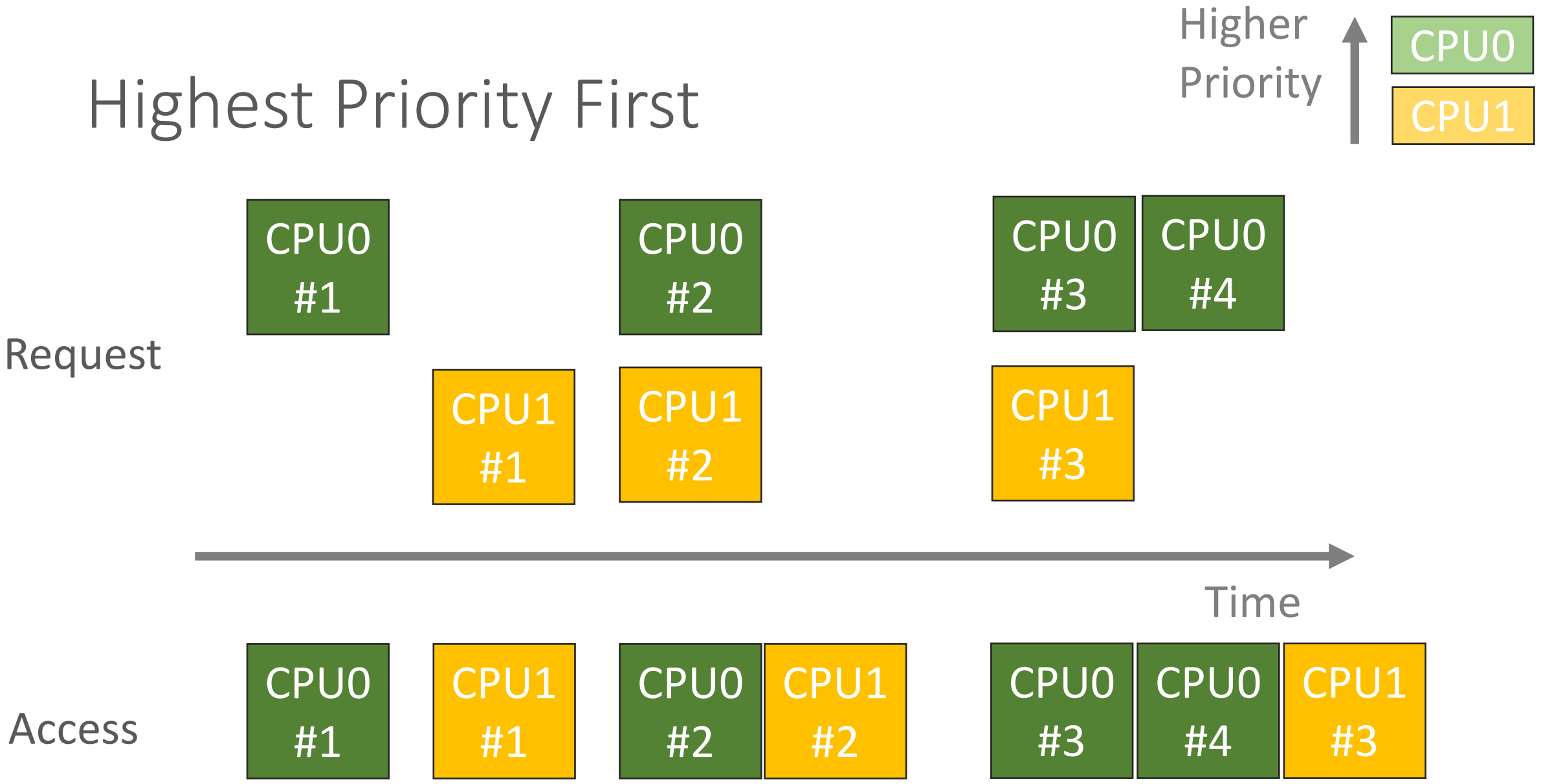


- Priority Fixed. Highest priority requester always wins.



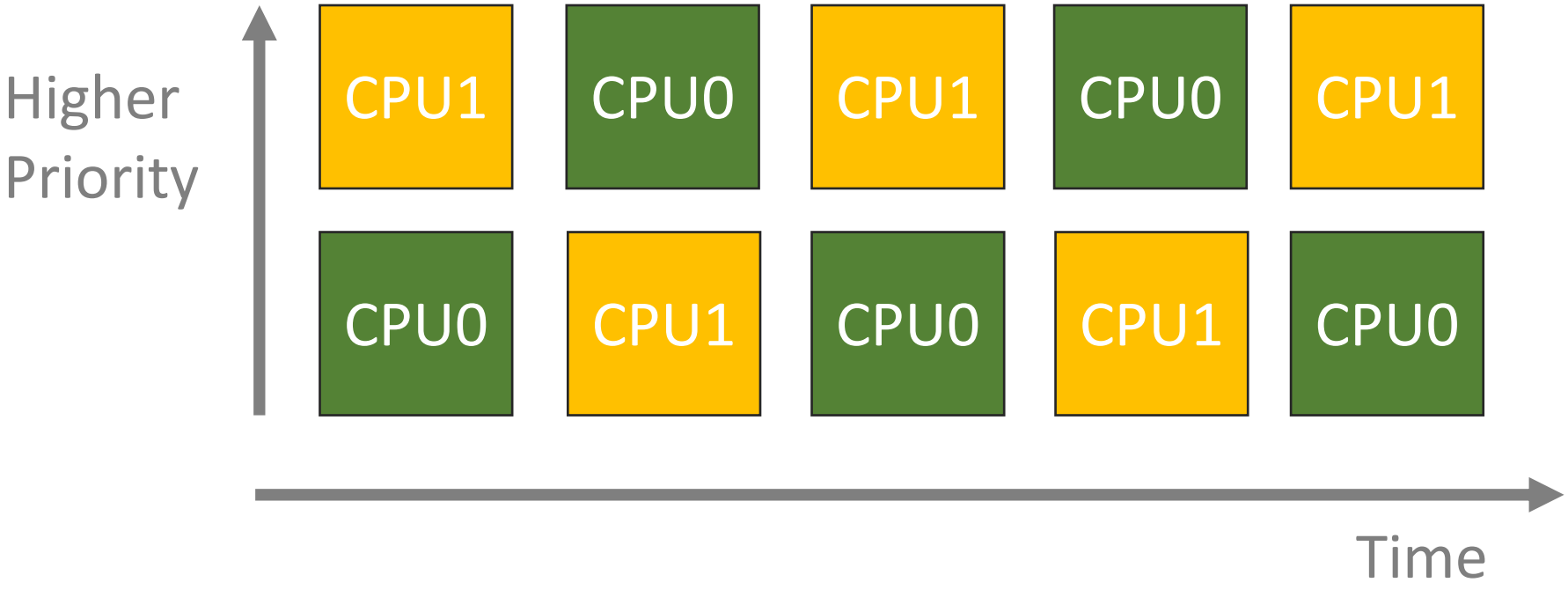
- Rules:
  - If CPU0 requests, CPU0 gets bus
  - Else if CPU0 does not request and CPU1 requests, CPU1 gets bus
  - Else bus idle

Highest Priority First

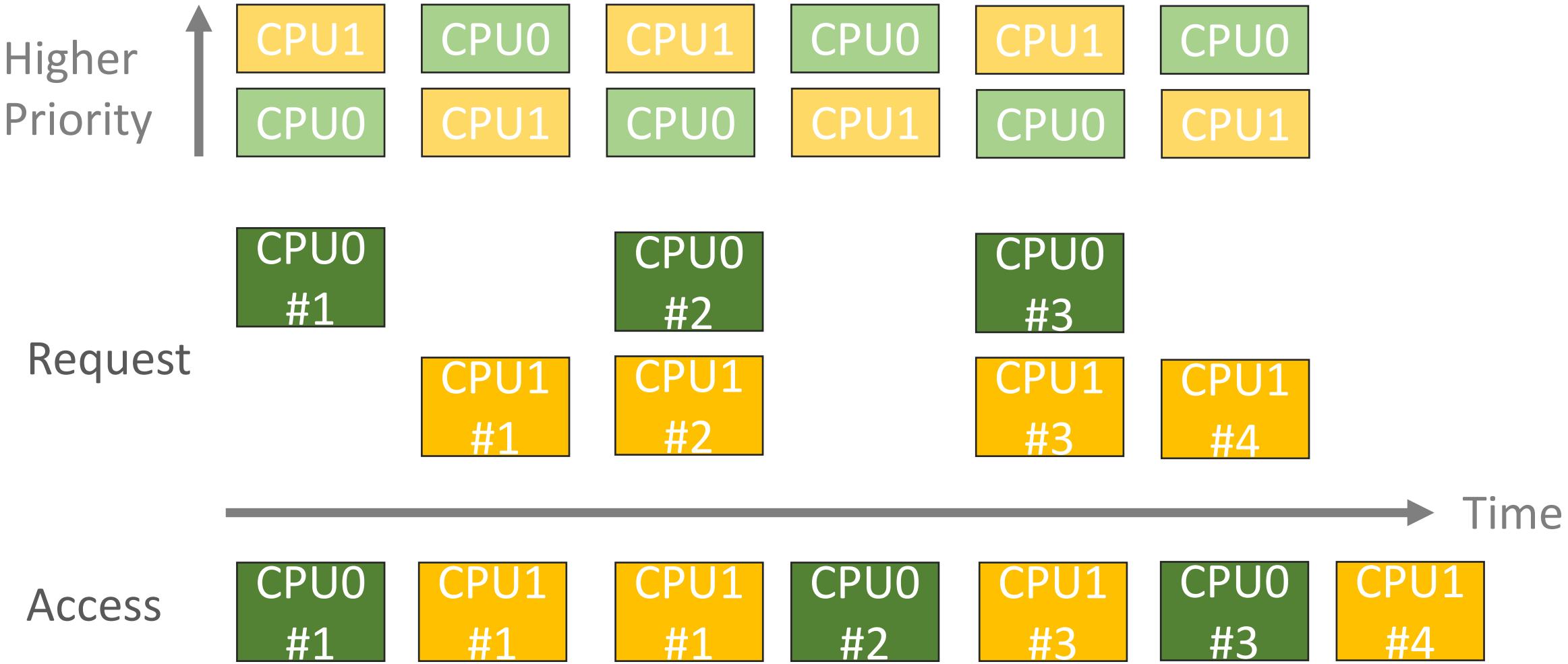


Round Robin

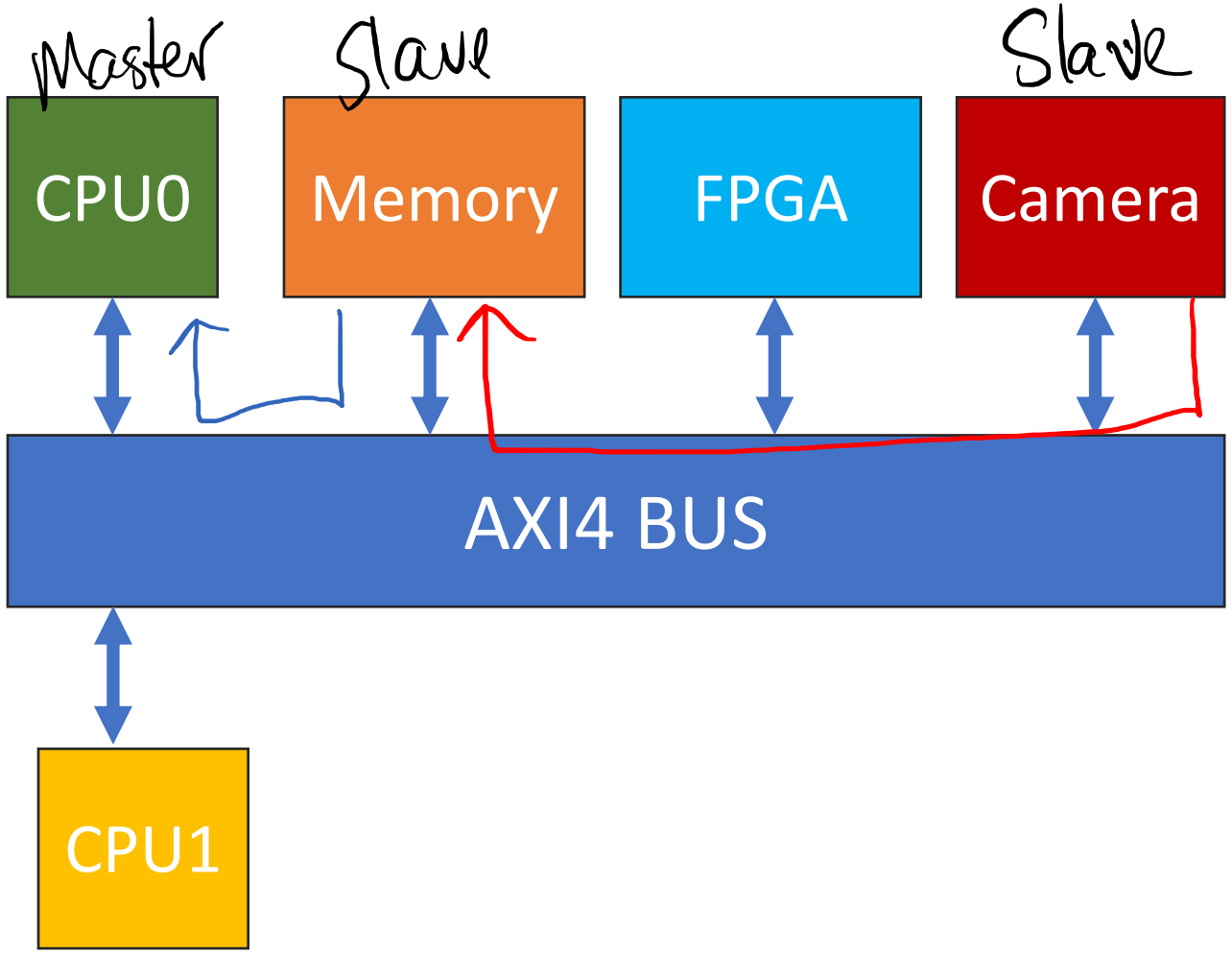
- Priority updates every cycle. Everyone get's equal access to highest priority



Round Robin


Higher Priority
CPU1
CPU0
CPU1
CPU0
CPU1
CPU0
CPU0
CPU1
CPU0
CPU1
CPU0
CPU1
Request
CPU0 #1
CPU0 #2
CPU0 #3
CPU1 #1
CPU1 #2
CPU1 #3
CPU1 #4
Time
Access
CPU0 #1
CPU1 #1
CPU1 #1
CPU0 #2
CPU1 #3
CPU0 #3
CPU1 #4

Q: How do I move data between the Camera and Memory?



A: The CPU copies data from Camera to Memory

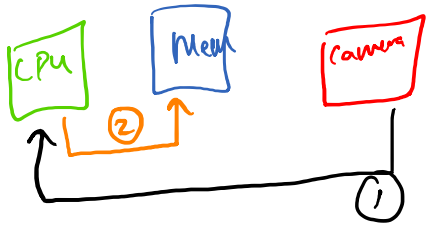
```
#define CAMERA_MMIO_ADDR 0x40000004
volatile uint32_t * camera =
        (uint32_t *)(CAMERA_MMIO_ADDR);
#define BUF_SIZE 1024;
uint32_t buf[BUF_SIZE];

int main () {

    while (true){
        copy_image(camera, buf, BUF_SIZE);
        detect_face(buf);
    }
}
```

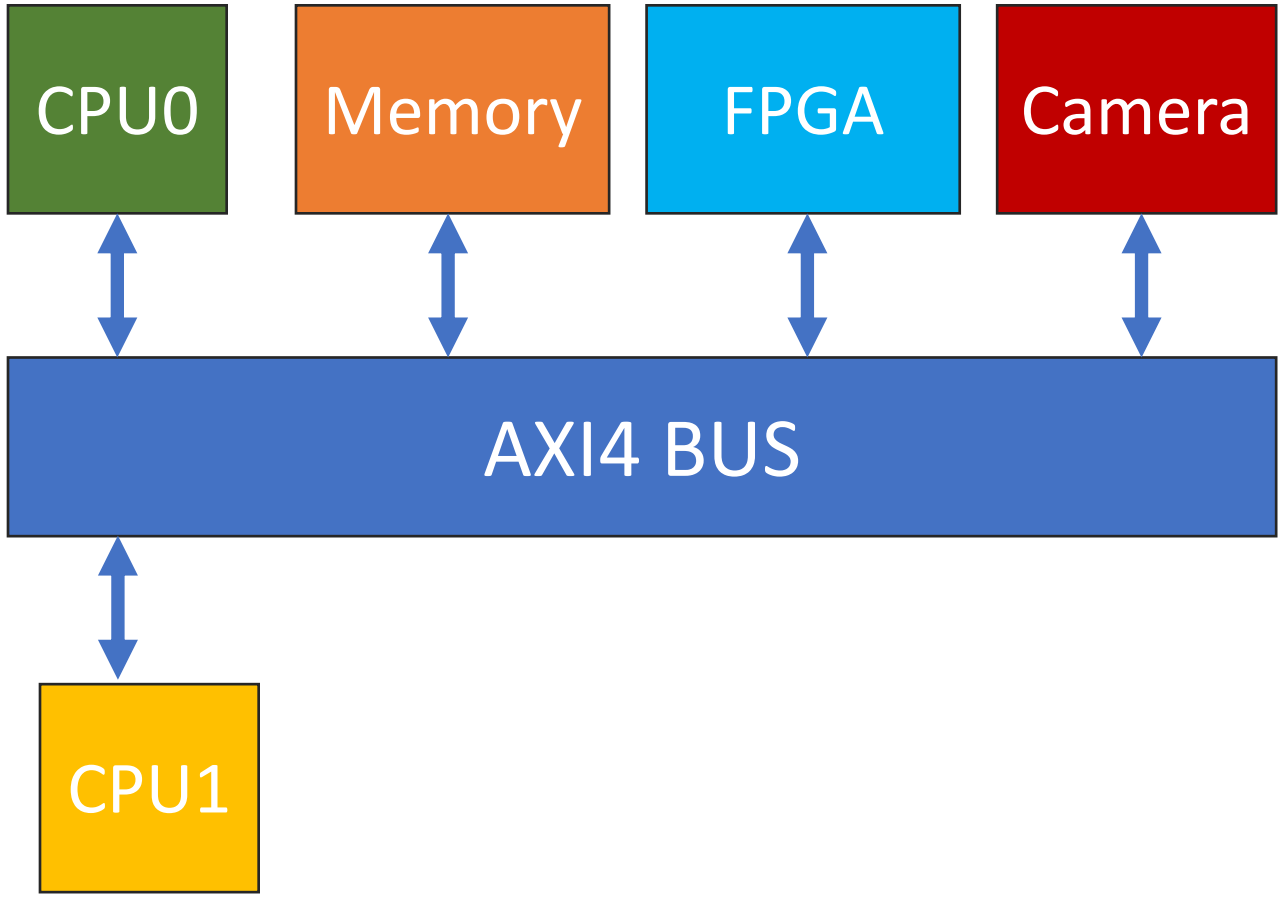
```
void copy_image (uint32_t * from,
                 uint32_t * to,
                 uint32_t size)
{
    register uint32_t reg;

    for (int i = 0; i < size; ++i){
    1 reg = *from;
    2 to[i] = reg;
    }4
}
```



1 load
2 store
3 increment
4 branch

What bus transactions result?



What else can the CPU do while copying data?

CPU 1 GHz → 1B cycles/second → 1B instructions/second

What else can the CPU do while copying data?

- CPU can do 1B instructions/second. (1GHz)
- 4 Instructions per loop
  - 1 load, 1 store, 1 increment, 1 branch
- 250M copies/second

4K Video: 1697 Mbps* = 212 MB / second

~85% CPU utilization for Copy!

What about Ethernet?

- CPU can do 1B instructions/second. (1GHz)
- 4 Instructions per loop
  - 1 load, 1 store, 1 increment, 1 branch
- 250M copies/second

- 1Gbps Ethernet:
- 1 Gbps Receive + 1Gbps Transmit = 2 Gbps
- 2Gbps = 250MB/second

- **Nothing. ~100% of CPU required?**

What if we do the copy on CPU1?

```
int main () {

    while (true){

        ask_cpu1_to_copy_image(camera, buf, BUF_SIZE);



        detect_face(buf);
    }
}
```

What if we do the copy on CPU1?

buf 2

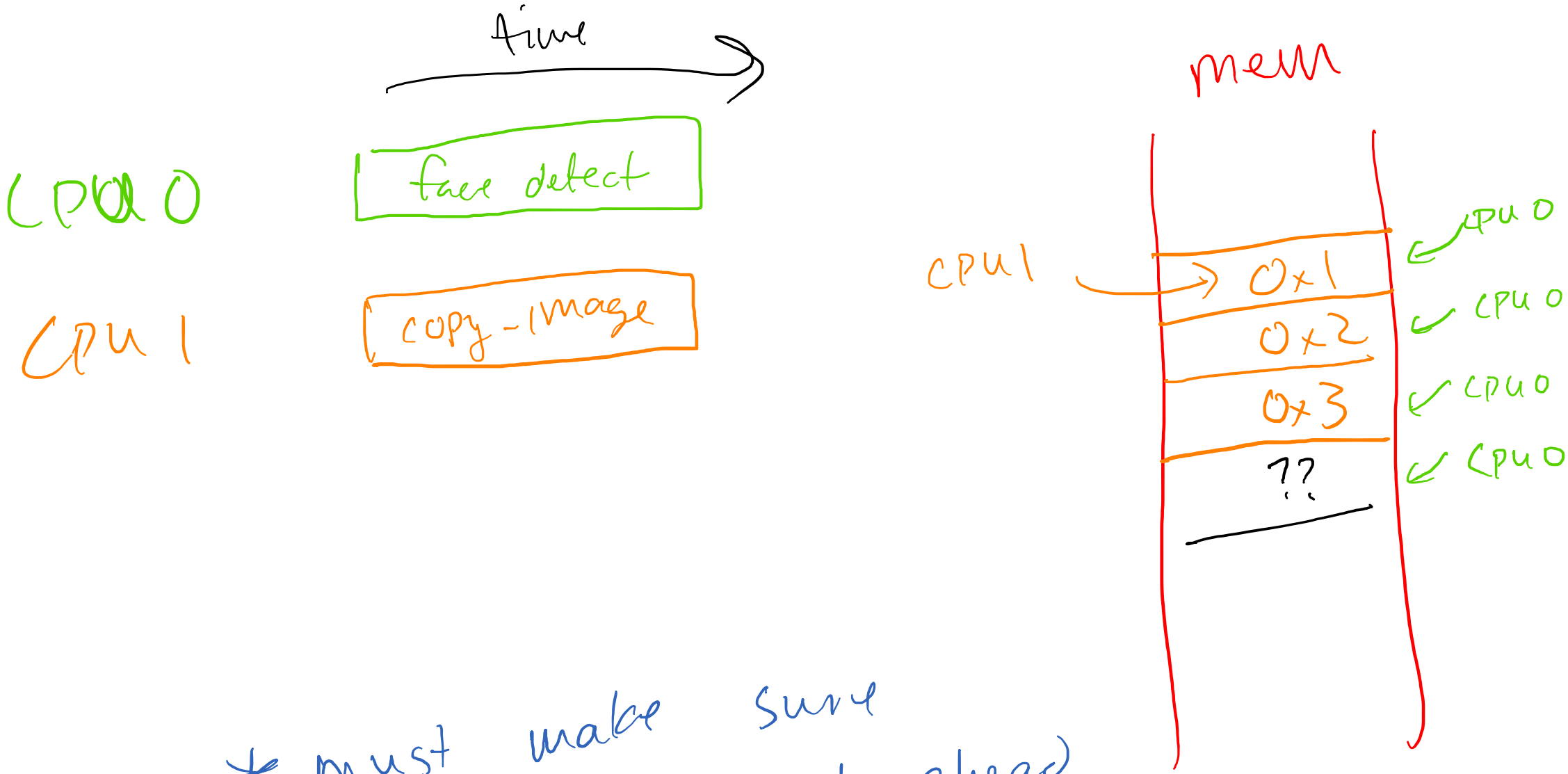
```
int main () {

    while (true){

        ask_cpu1_to_copy_image(camera, buf, BUF_SIZE);

        wait_for_cpu1_done();
        ask_cpu1_to_copy_image(camera, buf2, Buf_size);
        detect_face(buf);
    }
    wait_for_cpu1_done();
    detect_face(buf2);
```

4



* must make sure CPU 0 never gets ahead of CPU 1!

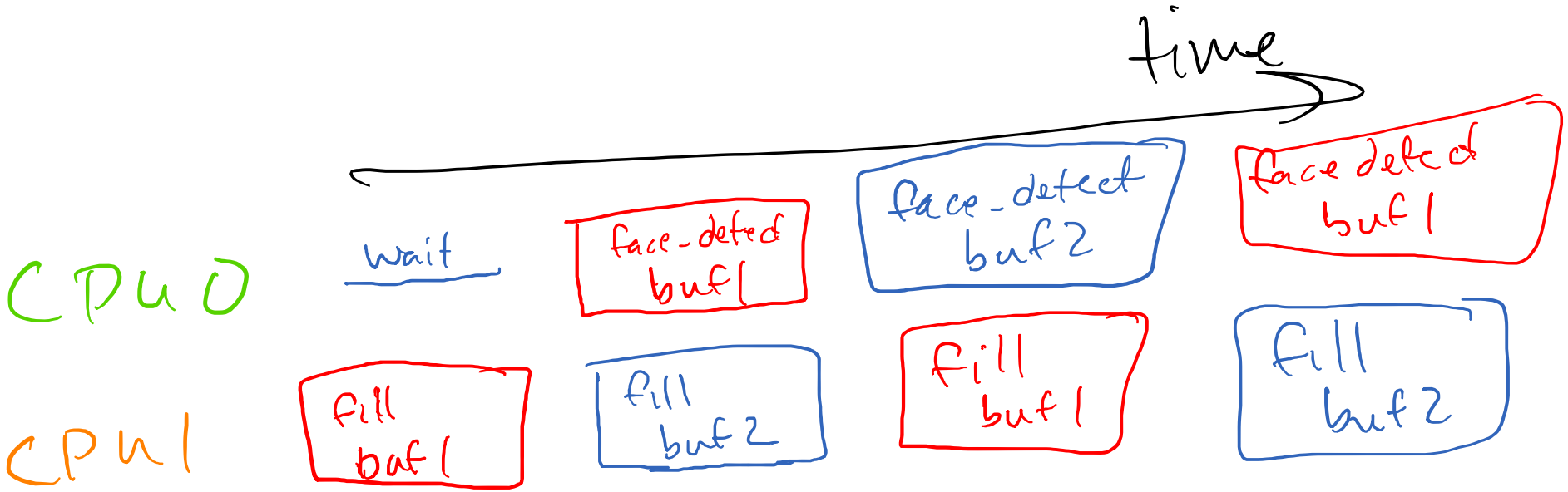
Copy on CPU1, Version 2.

```
int main () {

    ask_cpu1_to_copy_image(camera, buf1, BUF_SIZE);
    wait_for_cpu1_done();

    while (true){
        ask_cpu1_to_copy_image(camera, buf2, BUF_SIZE);
        detect_face(buf1);
        wait_for_cpu1_done();

        ask_cpu1_to_copy_image(camera, buf1, BUF_SIZE);
        detect_face(buf2);
        wait_for_cpu1_done();
    }
}
```


time
CPU 0
CPU 1
wait
face-detect buf1
face-detect buf2
face detect buf1
fill buf1
fill buf2
fill buf1
fill buf2

Why are we wasting a CPU do this?

```
void copy_image (uint32_t * from,
                 uint32_t * to,
                 uint32_t size)
{
   register uint32_t reg;

    for (int i = 0; i < size; ++i){

        reg = *from;

        to[i] = reg;
    }
}
```

DMA: Direct Memory Access

- A mini-CPU that does copy for you:

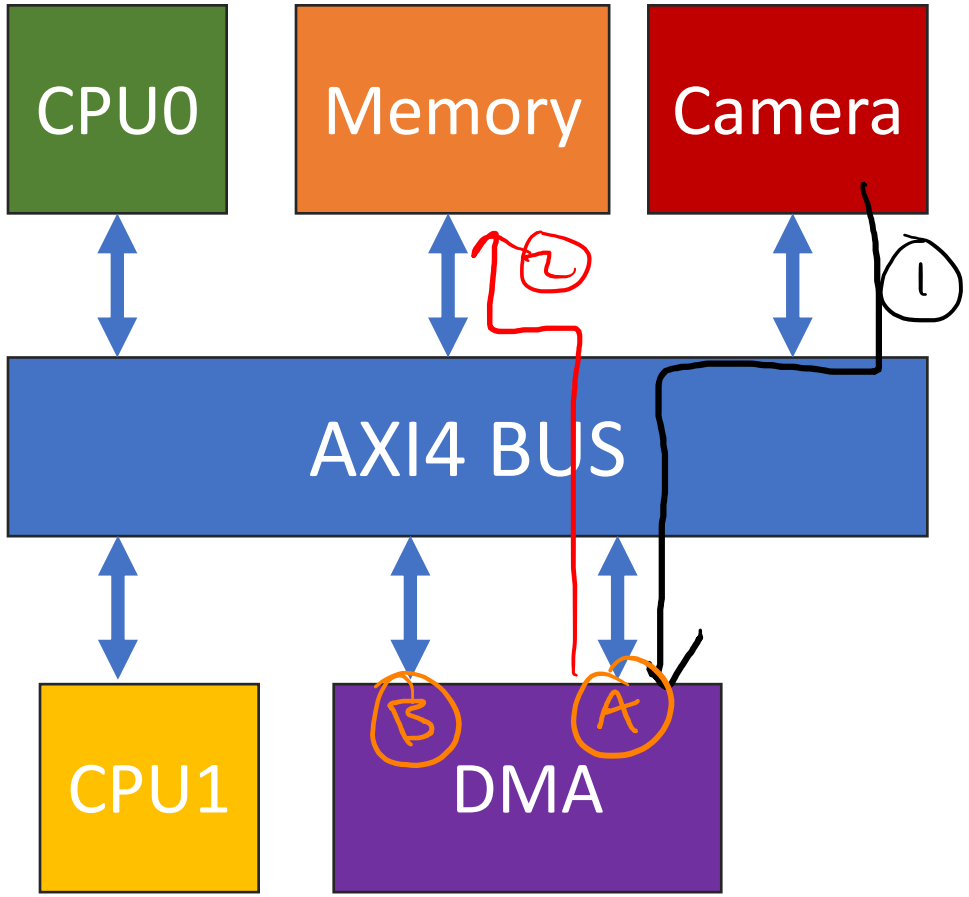
```
void copy (uint32_t * from,
               uint32_t * to,
               uint32_t size)
{
   register uint32_t reg;

    for (int i = 0; i < size; ++i){

        reg = *from;

        to[i] = reg;
    }
}
```

Why 2 interfaces?



- Interface ~~1~~ A: Copy Memory
  - Data-Intensive Interface
- Interface ~~2~~ B: Tell DMA <u>what</u> to copy
  - Control Interface

1 load to DMA
2 Store from DMA

DMA

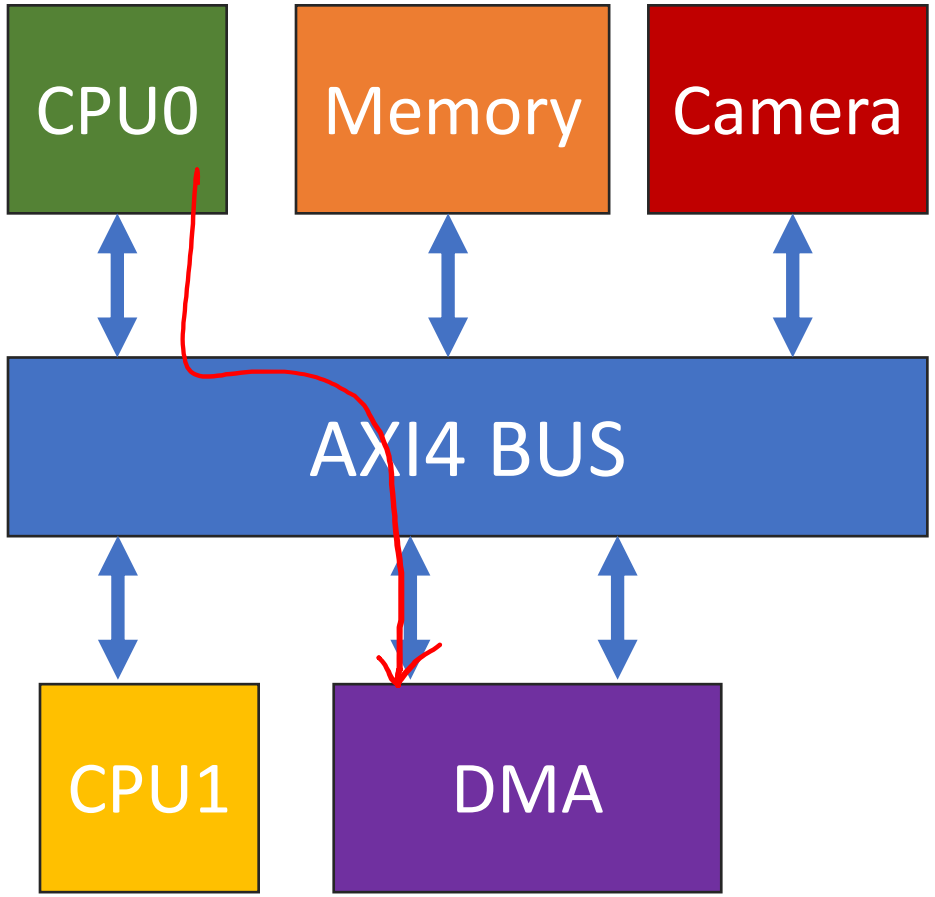
```
int main () {

    dma_copy_image(camera, buf1, BUF_SIZE);
    wait_for_dma_done();

    while (true){
        dma_copy_image(camera, buf2, BUF_SIZE);
        detect_face(buf1);
        wait_for_dma_done();

        dma_copy_image(camera, buf1, BUF_SIZE);
        detect_face(buf2);
        wait_for_dma_done();
    }
}
```

DMA signaling

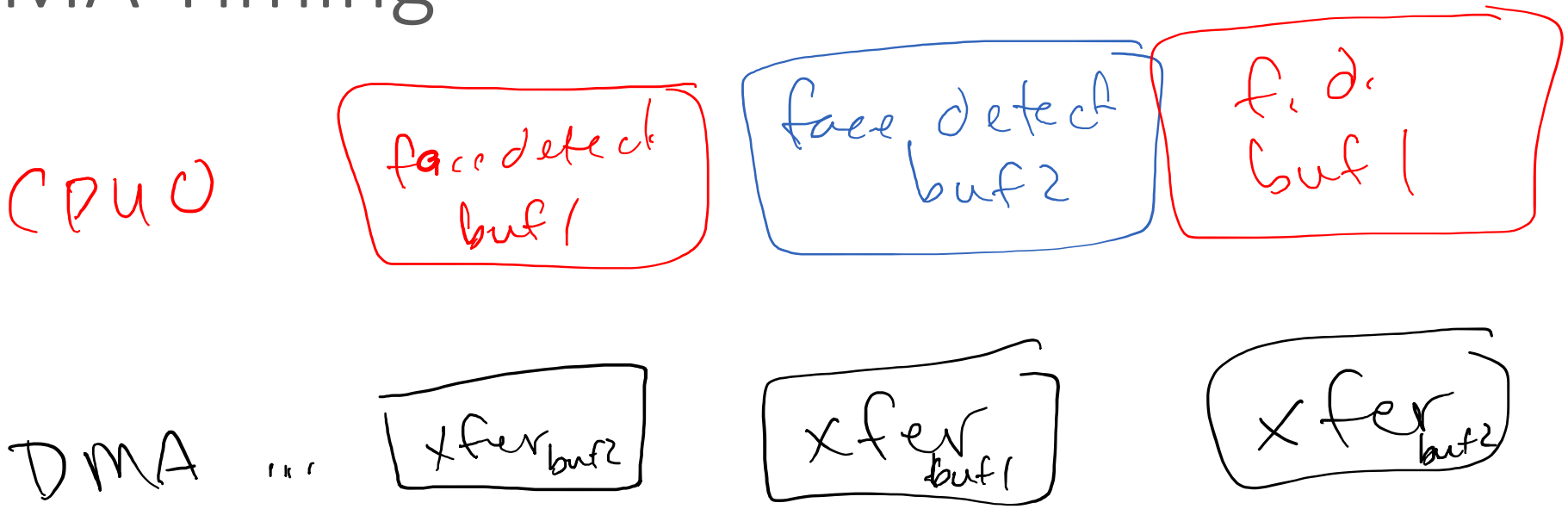


How do you know DMA done?

→ Ask MMIO Addr (This class)

→ Interrupt

DMA Timing


CPU0
face detect buf1
face detect buf2
f. d. buf1
DMA ...
xfer buf2
xfer buf1
xfer buf2

References

- https://vashivisuals.com/4k-beyond-video-data-rates/

05: Direct Memory Access

Engr 315: Hardware / Software Codesign

Andrew Lukefahr
Indiana University